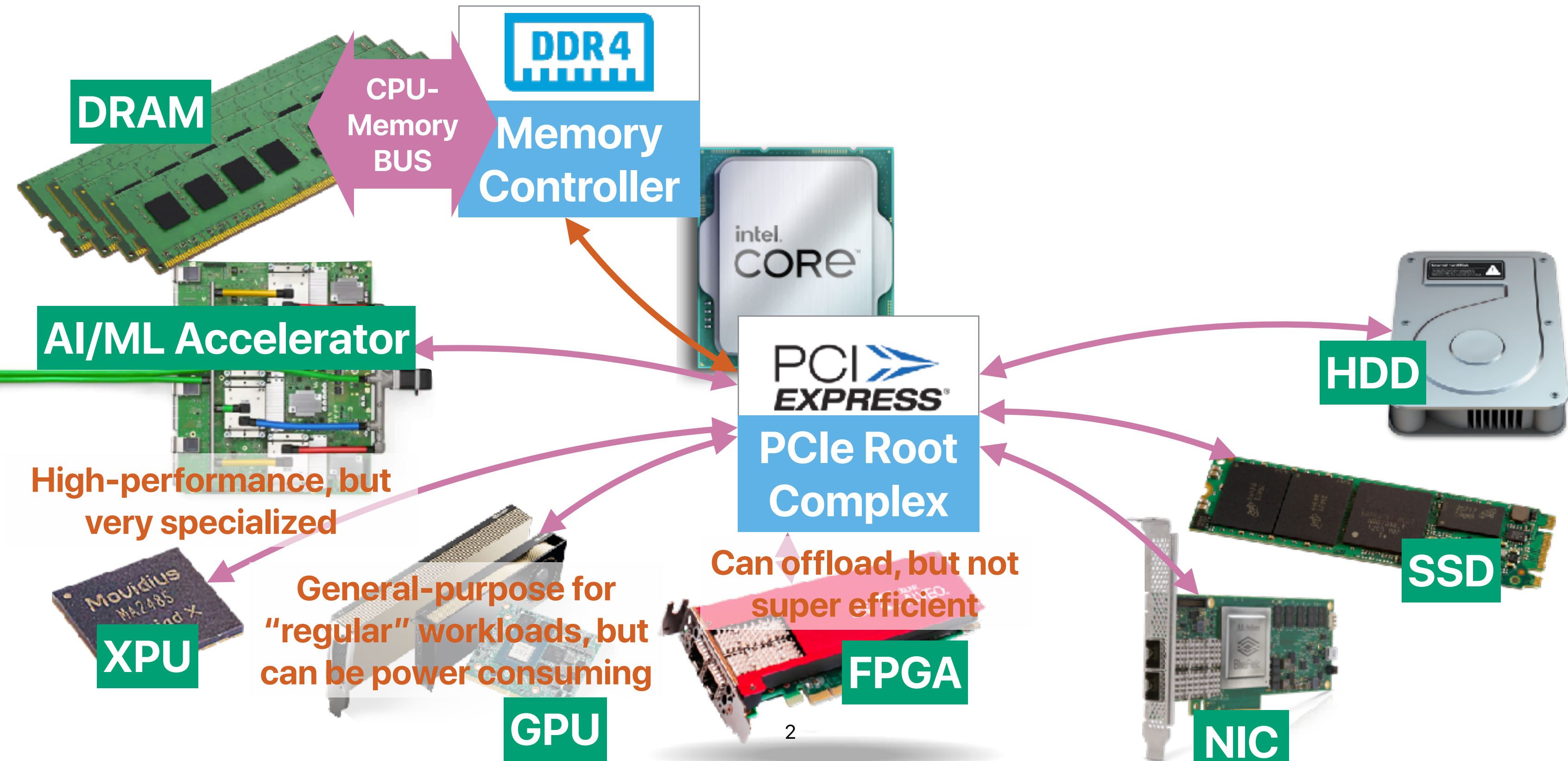


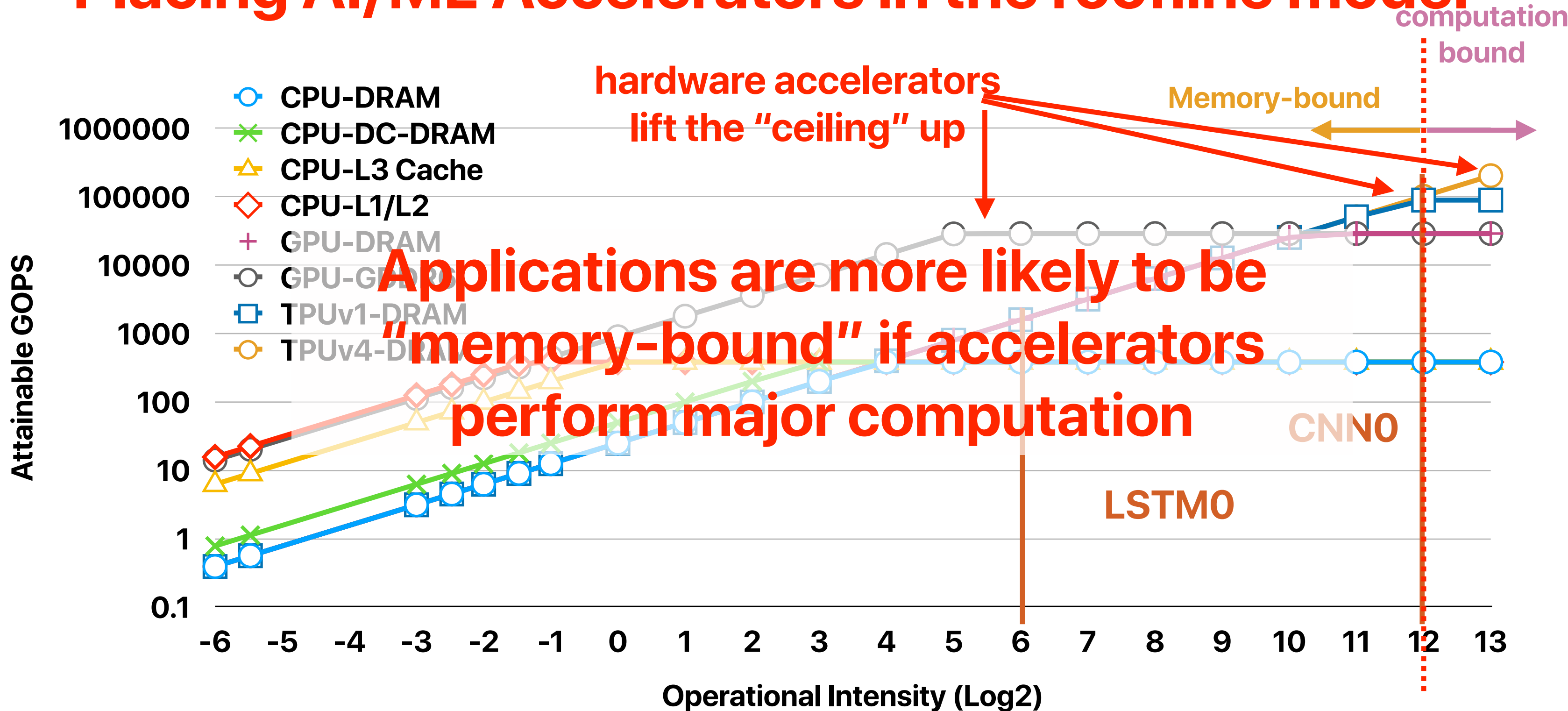
Memory subsystem (2)

Hung-Wei Tseng

Recap: The landscape of modern computers



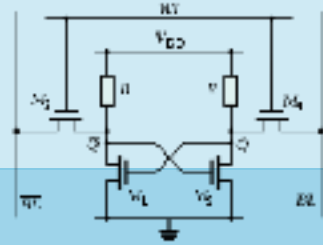
Placing AI/ML Accelerators in the roofline model



Recap: Memory technologies we have today

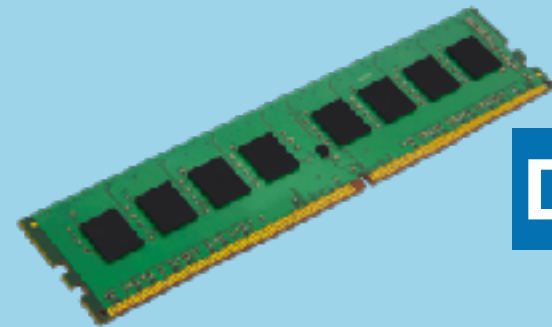
Volatile Memory

100ps



SRAM

ns

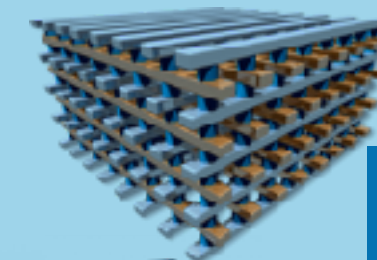


DRAM

us

ms

Non-Volatile Memory

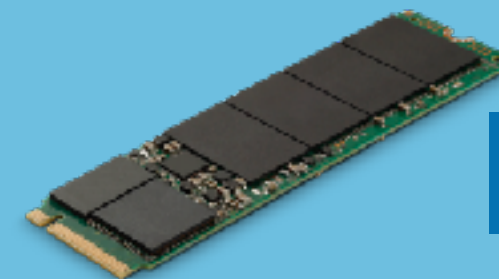


RRAM



PCM

3D XPoint



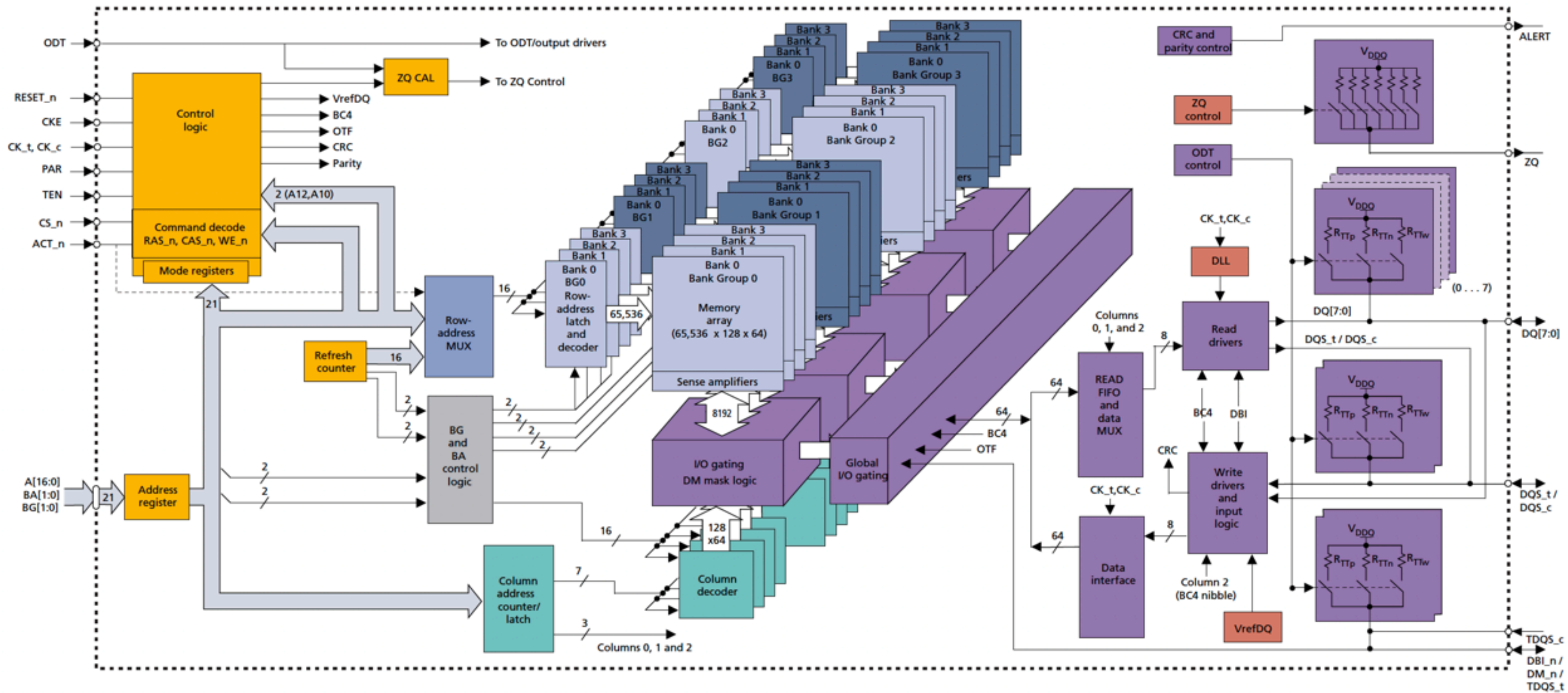
Flash memory



Hard Disk Drives

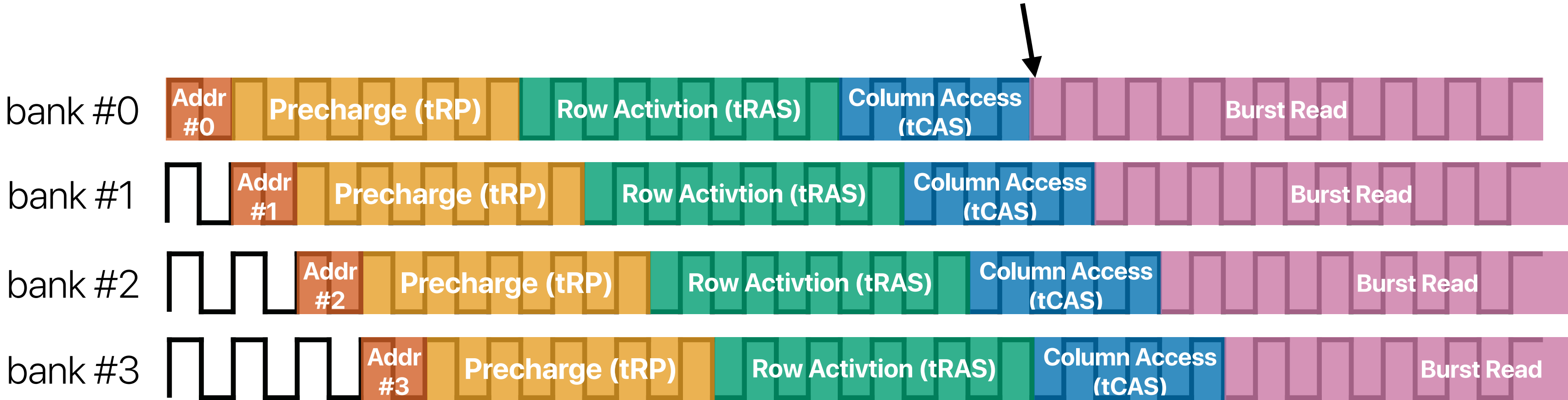
Recap: Improving memory-bounded applications

- Faster memory technologies
- Higher memory bandwidth
- Lower data volume



Recap: Multi-bank access

we can start output a "byte" from every 8 chips each cycle after this

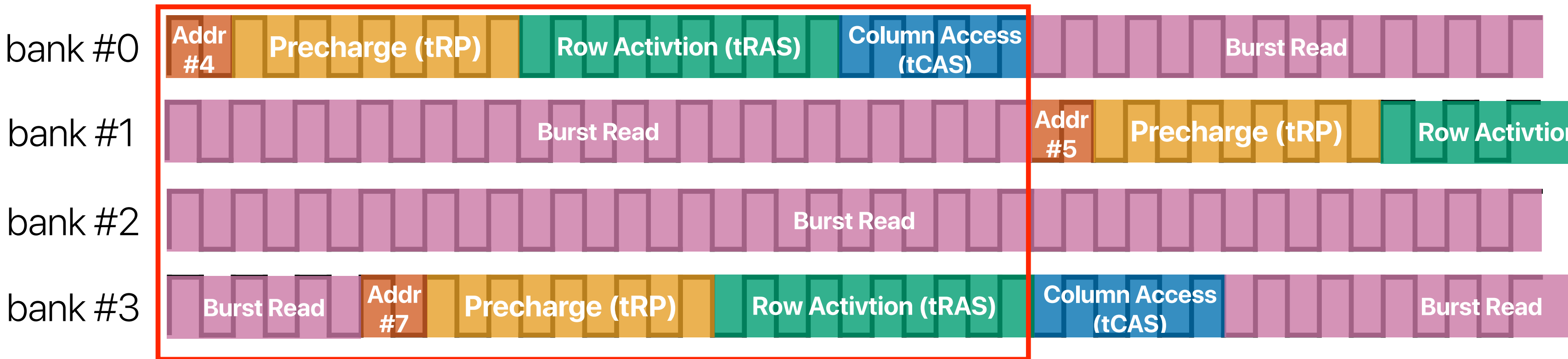


only one bank can accept request each cycle

the memory bandwidth
can be fully utilized after
this

Multi-bank access

The latency of pre-charge, row/column accesses is fully covered!



Outline

- Improve the performance of main memory subsystem
 - Increasing bandwidth (cont.)
- Offload computing to lower the data exchange volume
 - Processing near memory
 - Processing in memory
 - Processing using memory

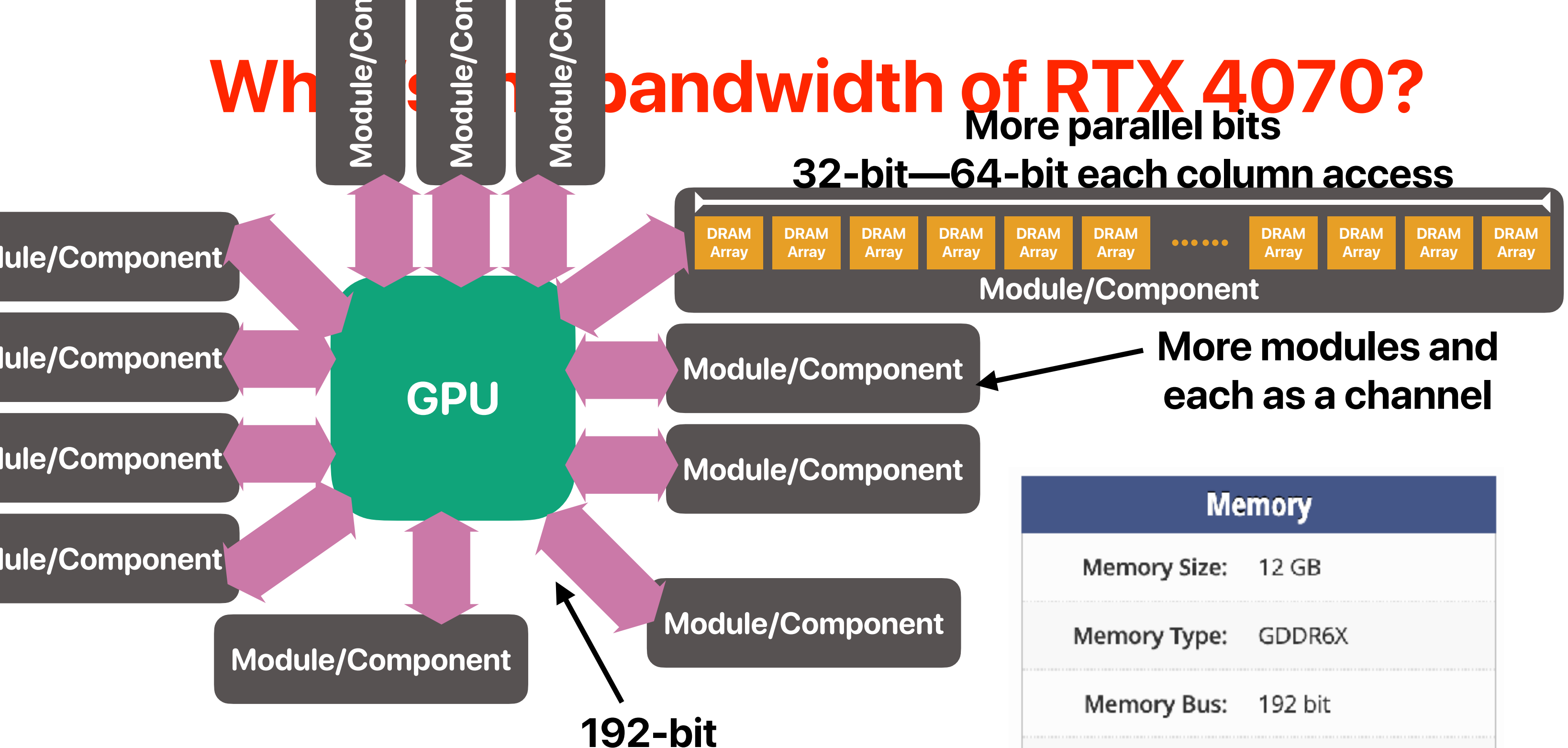
What's the bandwidth of RTX 4070?

	GeForce RTX 4070 Ti SUPER	GeForce RTX 4070 Ti	GeForce RTX 4070 SUPER	GeForce RTX 4070
GPU Engine Specs:				
NVIDIA CUDA® Cores	8448	7680	7168	5888
Boost Clock (GHz)	2.61	2.61	2.48	2.48
Base Clock (GHz)	2.34	2.31	1.98	1.92
Memory Specs:				
Standard Memory Config	16 GB GDDR6X	12 GB GDDR6X	12 GB GDDR6X	12 GB GDDR6X
Memory Interface Width	256-bit	192-bit	192-bit	192-bit

Why is the bandwidth of RTX 4070?

More parallel bits

32-bit—64-bit each column access



More modules and each as a channel

192-bit

Memory	
Memory Size:	12 GB
Memory Type:	GDDR6X
Memory Bus:	192 bit
Bandwidth:	504.2 GB/s

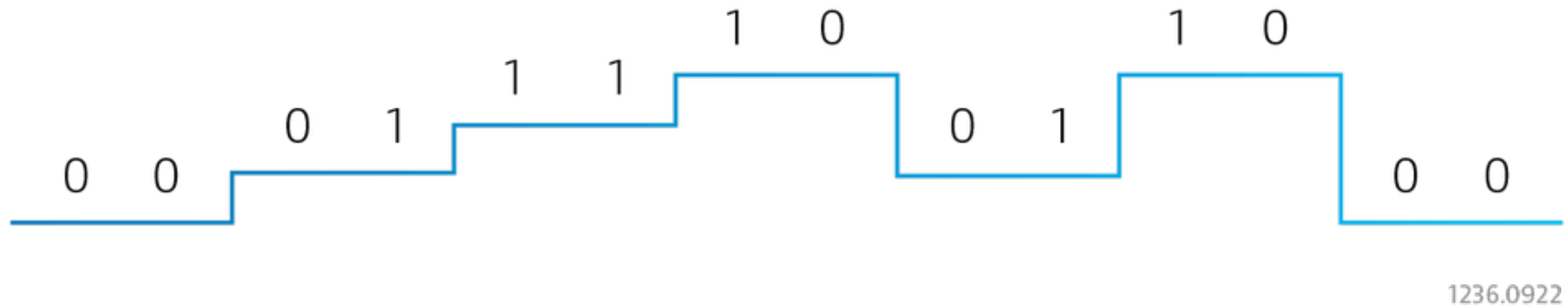
If the bus is **192**-bit (24B) wide, bandwidth is

$$\frac{192}{8} \times 10501 \times 10^6 = 252024 MB/sec = 252 GB/sec$$

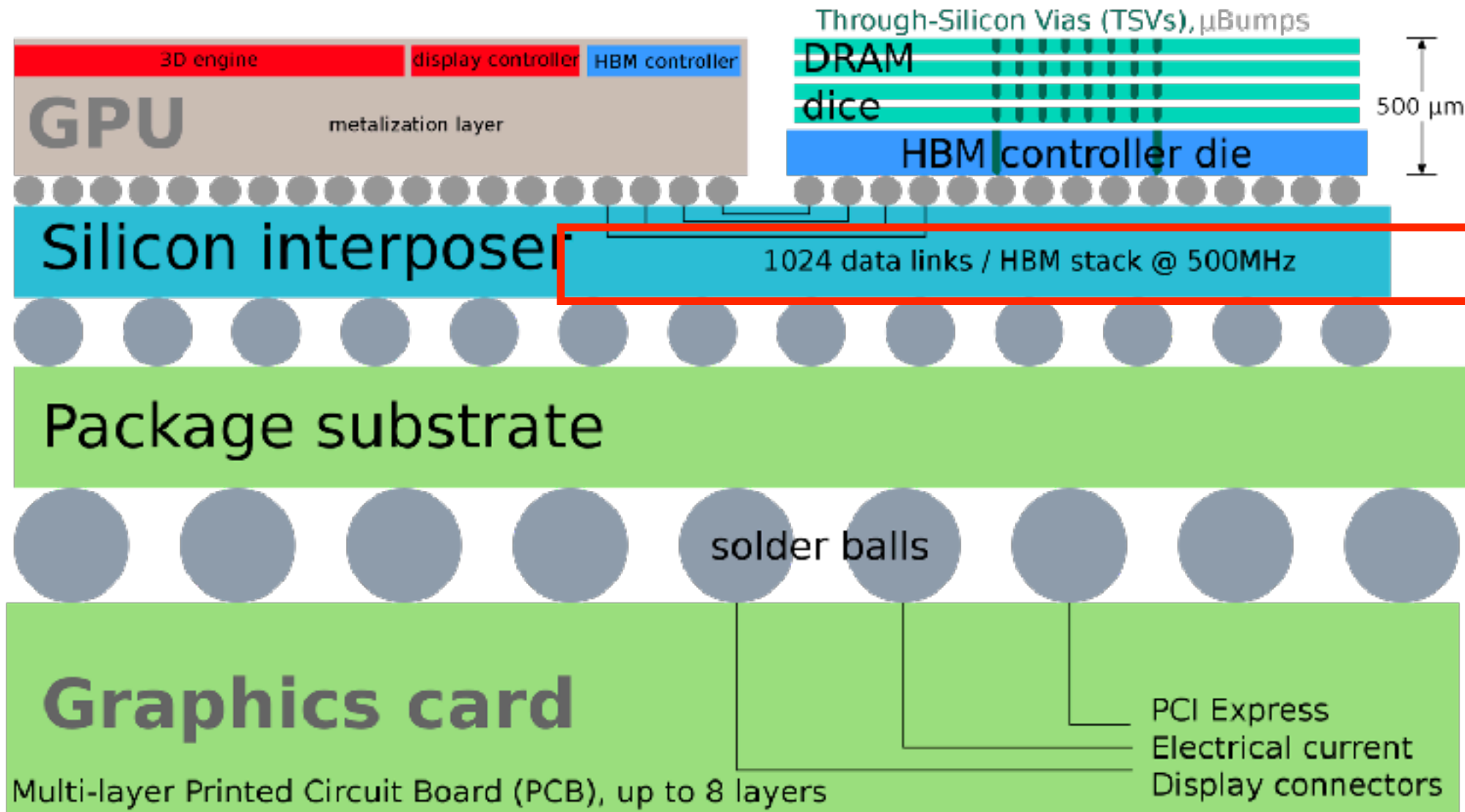
<https://www.techpowerup.com/gpu-specs/geforce-rtx-4070.c3924>

Signaling in DDR6X

PAM4 Signaling Technology



HBM (High Bandwidth Memory)



$$0.5G \times \frac{1024bits}{8bits} = 64GB/sec$$

If you have 4x modules —
 $4 \times 64 = 256 \text{ GB/sec}$

**HBM2 increase the clock
rate to 2GHz — 1TB/sec**

What's the pros & cons of GDDR v.s. HBM

GDDR v.s. HBM

GDDR v.s. HBM

- HBM's bandwidth is wider
- HBM is more expensive — pin counts are more expensive
- HBM is limited by the per-chip heat dissipation — less powerful

Case: what's the goal of the following program and what do you expect the performance would change when size varies?

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/time.h>
#include <time.h> /* for clock_gettime */
#include <malloc.h>
```

```
void write_memory_loop(void* array, size_t size) {
    size_t* carray = (size_t*) array;
    size_t i;
    for (i = 0; i < size / sizeof(size_t); i++) {
        carray[i] = 1;
    }
}
```

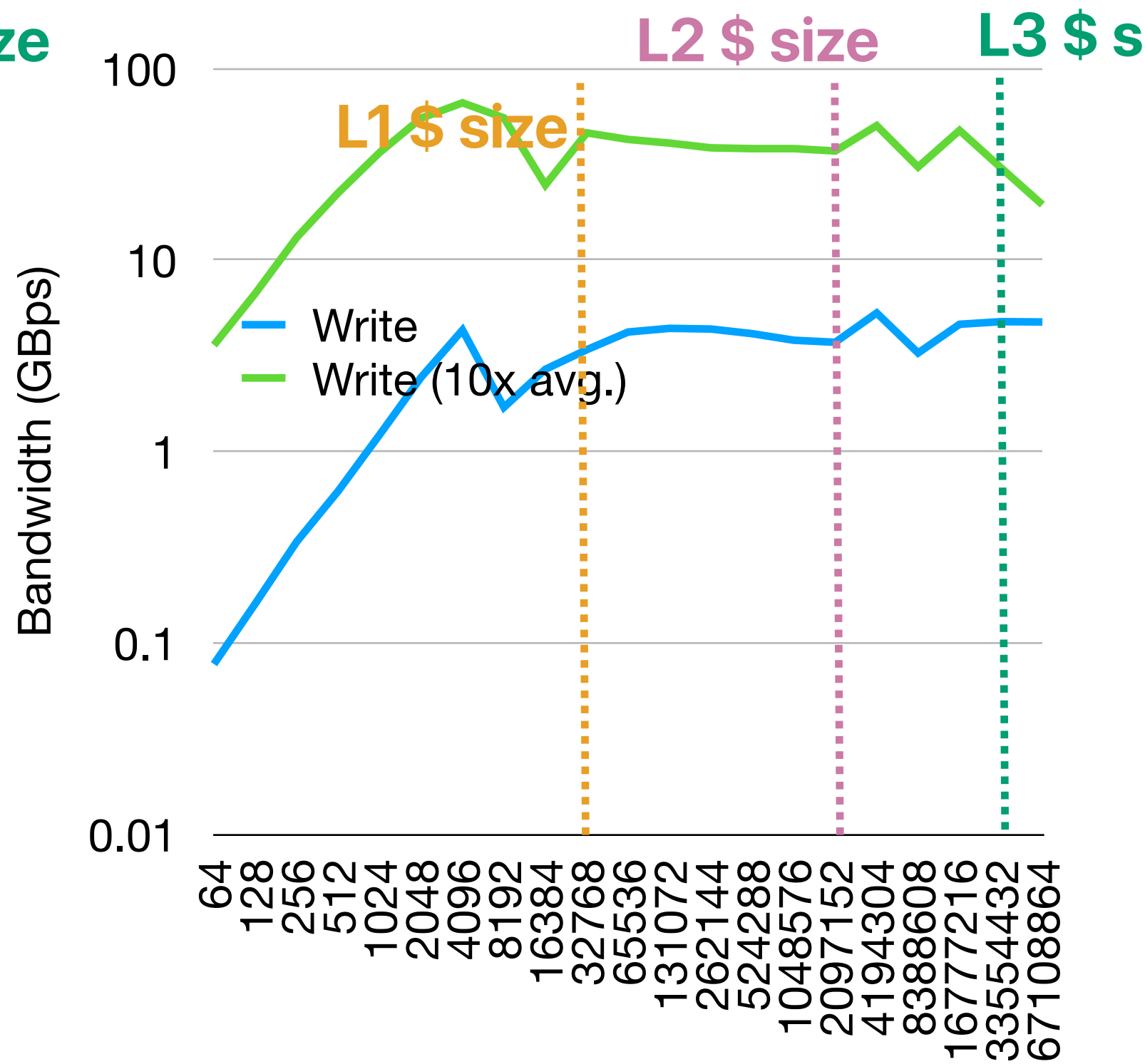
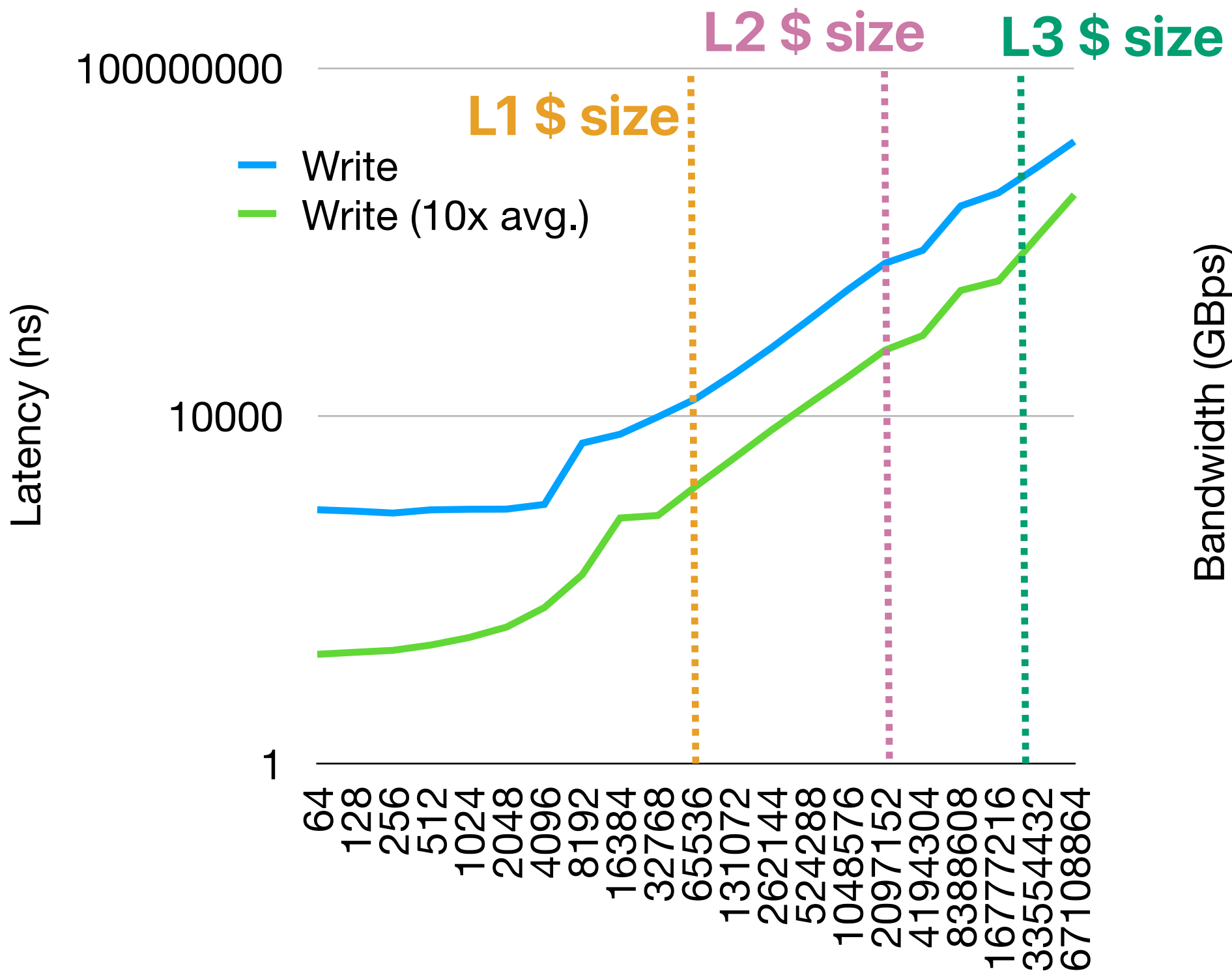
```
int main(int argc, char **argv)
{
    size_t *array;
    size_t *dest;
    size_t size;
    double total_time;
    struct timespec start, end;
    struct timeval time_start, time_end;
```

```
    size = atoi(argv[1])/sizeof(size_t);
    array = (size_t *)malloc(sizeof(size_t)*size);
    dest = (size_t *)malloc(sizeof(size_t)*size);
    clock_gettime(CLOCK_MONOTONIC, &start); /* mark start
time */

    write_memory_loop(array, size);

    clock_gettime(CLOCK_MONOTONIC, &start); /* mark start
time */
    clock_gettime(CLOCK_MONOTONIC, &end); /* mark start
time */
    total_time = ((end.tv_sec * 1000000000.0 +
end.tv_nsec) - (start.tv_sec * 1000000000.0 +
start.tv_nsec));
    fprintf(stderr, "Latency: %.0lf ns, GBps: %lf,
%llu\n", total_time, (double)((double)size*sizeof(size_t)/
(total_time)), dest[rand()%size]);
    return 0;
}
```

Performance Chart (on Intel Core i7 14700K)



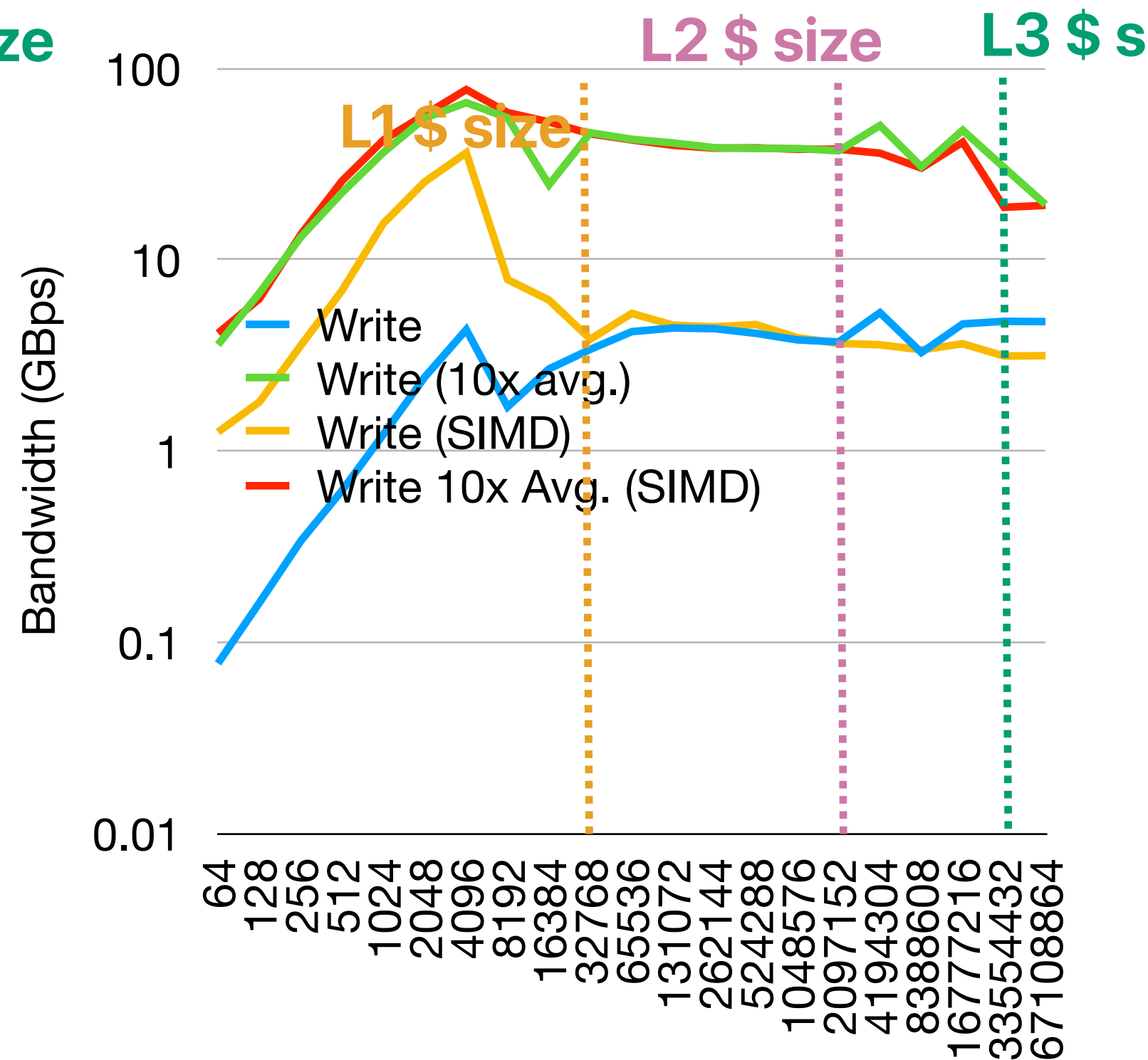
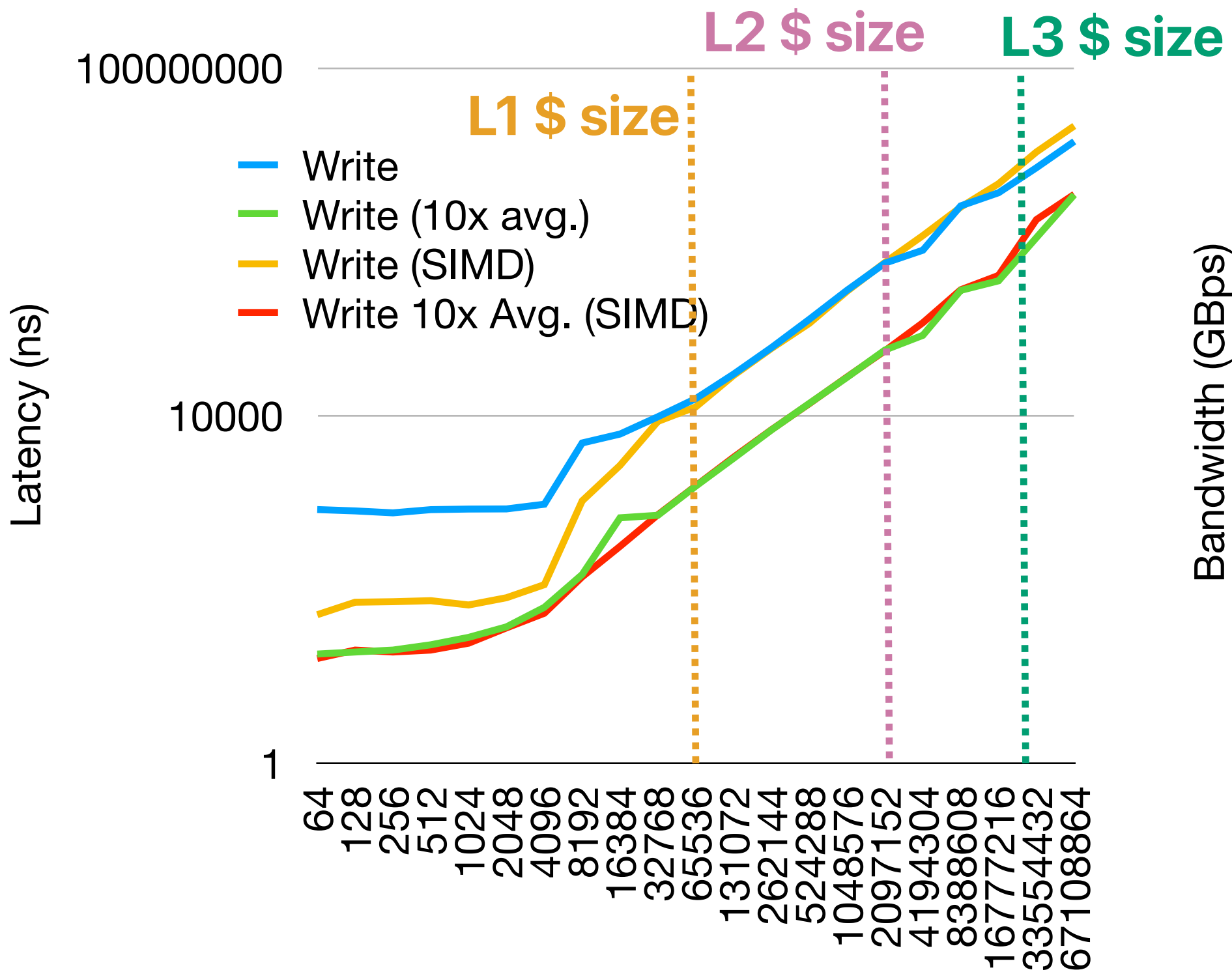
Compare these two versions, what are the differences and implications to performance?

```
void write_memory_avx(void* array, size_t size) {
    __m256i* varray = (__m256i*) array;

    __m256i vals = _mm256_set1_epi32(1);
    size_t i;
    for (i = 0; i < size / sizeof(__m256i); i++) {
        _mm256_store_si256(&varray[i], vals); // This will
generate the vmovaps instruction.
    }
}
```

```
void write_memory_loop(void* array, size_t size) {
    size_t* carray = (size_t*) array;
    size_t i;
    for (i = 0; i < size / sizeof(size_t); i++) {
        carray[i] = 1;
    }
}
```


Performance Chart (on Intel Core i7 14700K)



Case: what's the goal of the following program and what do you expect the performance would change when size varies?

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/time.h>
#include <time.h> /* for clock_gettime */
#include <malloc.h>

void write_memory_avx(void* array, size_t size) {
    __m256i* varray = (__m256i*) array;

    __m256i vals = _mm256_set1_epi32(1);
    size_t i;
    for (i = 0; i < size / sizeof(__m256i); i++) {
        _mm256_store_si256(&varray[i], vals); // This will
generate the vmovaps instruction.
    }
}

int main(int argc, char **argv)
{
    size_t *array;
    size_t *dest;
    size_t size;
```

```
double total_time;
struct timespec start, end;
struct timeval time_start, time_end;
size = atoi(argv[1])/sizeof(size_t);
array = (size_t *)malloc(sizeof(size_t)*size);
dest = (size_t *)malloc(sizeof(size_t)*size);
clock_gettime(CLOCK_MONOTONIC, &start); /* mark start
time */

    write_memory_avx(array, size);

    clock_gettime(CLOCK_MONOTONIC, &start); /* mark start
time */
    memcpy(dest, array, size*sizeof(size_t));
    clock_gettime(CLOCK_MONOTONIC, &end); /* mark start
time */
    total_time = ((end.tv_sec * 1000000000.0 +
end.tv_nsec) - (start.tv_sec * 1000000000.0 +
start.tv_nsec));
    fprintf(stderr, "Latency: %.0lf ns, GBps: %lf,
%llu\n", total_time, (double)((double)size*sizeof(size_t)/
(total_time)), dest[rand()%size]);
    return 0;
}
```

The program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/time.h>
#include <time.h> /* for clock_gettime */
#include <malloc.h>

void write_memory_avx(void* array, size_t size) {
    size_t* carray = (size_t*) array;
    size_t i;
    for (i = 0; i < size / sizeof(size_t); i++) {
        carray[i] = 1;
    }
}

int main(int argc, char **argv)
{
    size_t *array;
    size_t *dest;
    size_t size;
    double total_time;
    struct timespec start, end;
    struct timeval time_start, time_end;
    size = atoi(argv[1])/sizeof(size_t);
    array = (size_t *)malloc(sizeof(size_t)*size);
```

```
    dest = (size_t *)malloc(sizeof(size_t)*size);
    clock_gettime(CLOCK_MONOTONIC, &start); /* mark start
time */

    write_memory_avx(array, size);

    clock_gettime(CLOCK_MONOTONIC, &start); /* mark start
time */
    memcpy(dest, array, size*sizeof(size_t));
    clock_gettime(CLOCK_MONOTONIC, &end); /* mark start
time */
    total_time = ((end.tv_sec * 1000000000.0 +
end.tv_nsec) - (start.tv_sec * 1000000000.0 +
start.tv_nsec));
    fprintf(stderr, "Latency: %.0lf ns, GBps: %lf,
%llu\n", total_time, (double)((double)size*sizeof(size_t)/
(total_time)), dest[rand()%size]);
    return 0;
}
```

memmove & memcpy: 5% cycles in Google's datacenter

Svilen Kanev, Juan Pablo Darago, Kim Hazelwood,
Parthasarathy Ranganathan, Tipp Moseley, Gu-Yeon Wei, and
David Brooks. ISCA '15

Why is memcpy so often?

Network protocol stack

How do applications(e.g., server/client) interpret data: HTTPS, FTP, SSH, RTSP ...



How do applications connect to each other (end-to-end reliability): TCP, UDP, RTP



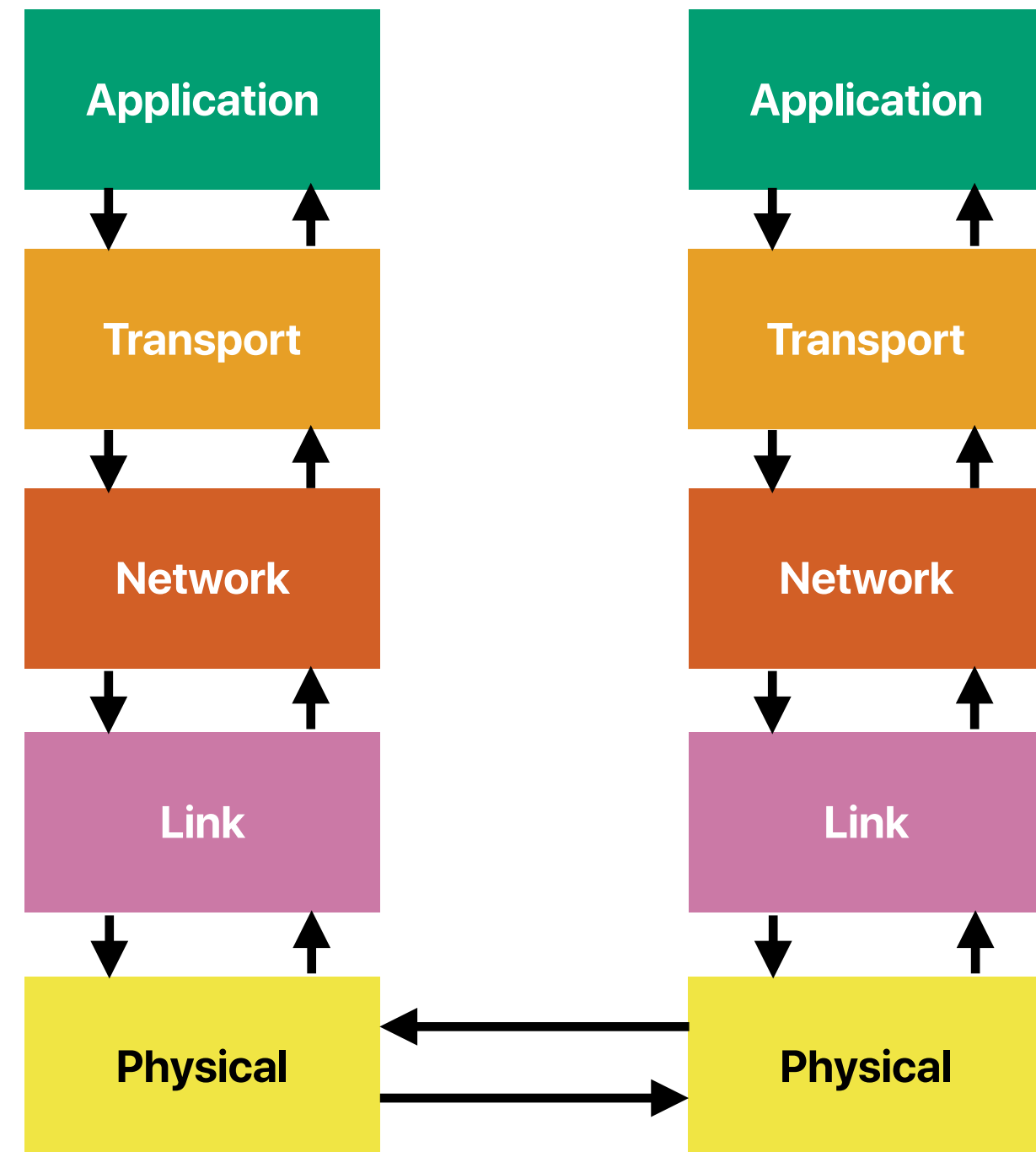
How do computers find each other: IP, ARP, ICMP



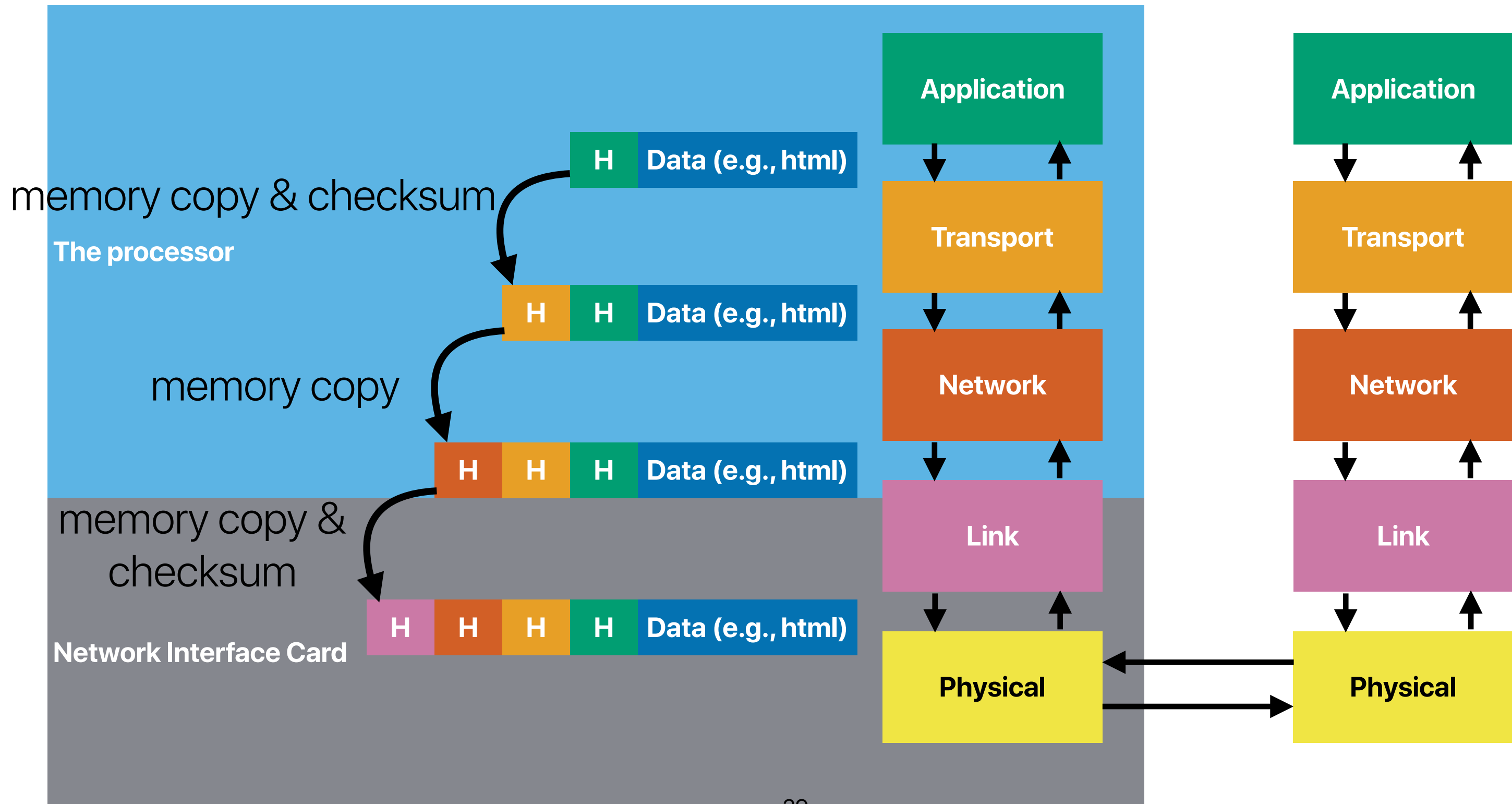
How do a computer use the media:
IEEE 802.3 (ethernet), IEEE 802.11 (WiFi)



How do the computer encode data on the media



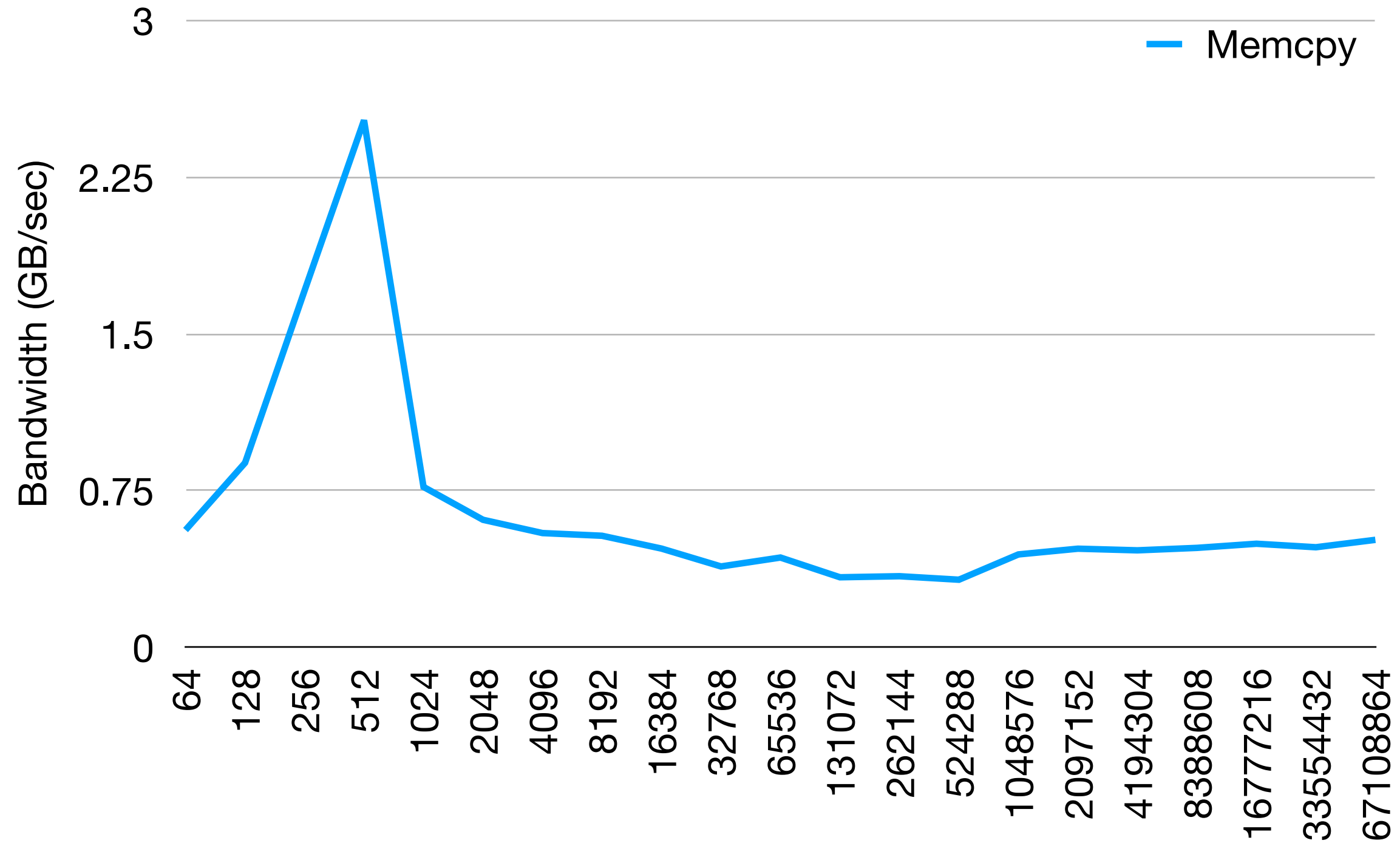
Network protocol stack



Use cases of memcpy

- Network packets (each layer needs to adjust the header and payload locations)
- Copy-on-write (spawning new process)

Memory copy bandwidth



Why is memcpy so slow?

Why is memcpy slow?

What's the x86 code for memcpy

memcpy in x86 by Bard

Regenerate draft



Here's a breakdown of the x86 code for memcpy, incorporating insights from various sources and addressing potential concerns:

Key Considerations:

- **Architecture and Optimization:** The exact code can vary depending on the specific x86 architecture (32-bit vs. 64-bit) and compiler optimizations.
- **Overlapping Memory:** `memcpy` assumes non-overlapping memory regions. For overlapping regions, use `memmove`.

Common Implementation (32-bit x86):

Code snippet

```
1 memcpy:
    push ebp
    mov ebp, esp

    push esi
    push edi
    push ecx

    mov edi, [ebp + 8] ; Destination address
    mov esi, [ebp + 12] ; Source address
    mov ecx, [ebp + 16] ; Number of bytes to copy

    rep movsb          ; Copy bytes using movsb instruction

    pop ecx
    pop edi
    pop esi

    pop ebp
    ret
```

Use code with caution. [Learn more](#)

Explanation:

1. Prologue:

- Saves registers `ebp`, `esi`, `edi`, and `ecx` on the stack.

2. Parameter Access:

- Loads function arguments (destination, source, and size) into registers.

3. Main Loop:

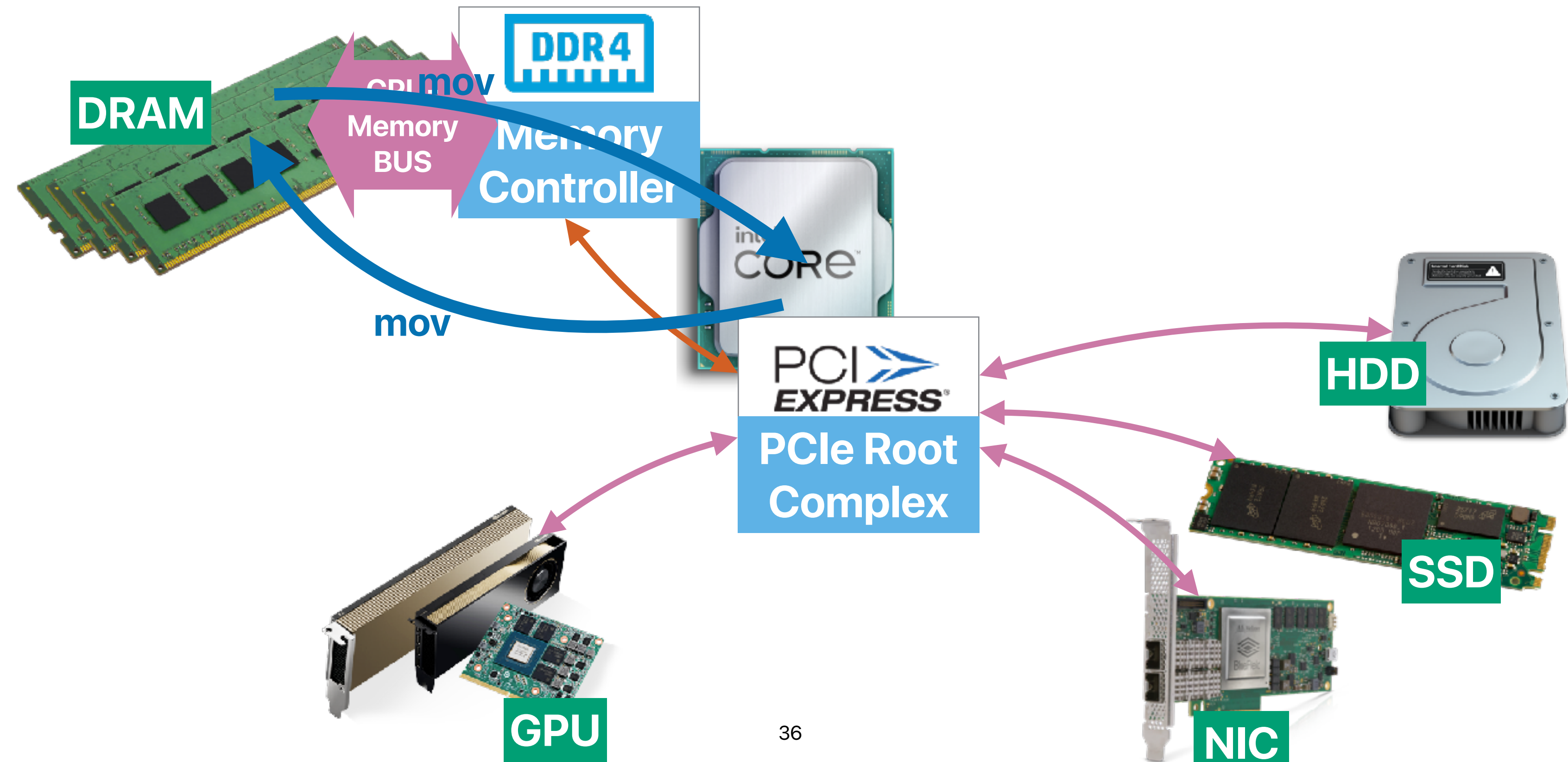
- `rep movsb`:
 - `rep`: Repeat until `ecx` becomes zero.
 - `movsb`: Move a byte from `esi` to `edi`.

4. Epilogue:

- Restores saved registers.
- Returns from the function.



Recap: The landscape of modern computers



Why is memcpy slow?

- Each instruction can only move limited amount of data
- Lots of instructions to move data — processor can only handle limited outstanding memory requests
- They must temporarily store data in CPU registers
- Data have to flow through CPU-memory bus twice
- Other overhead
 - Page fault

Implications

- The cost of load a word is a lot
 - More than just a load — you need to calculate the effective address
 - That's why we want AVX to load 256-bit (32B or 4 64-bit words) to load in one instruction
- The cost of a page fault is significant
 - That's why we see the first write/read is a lot longer
 - Huge page can be helpful
- Performance optimization in software is hard!!!

Electrical Computer Science Engineering

277

つくづく

